

Conceitos e importância do reuso de software

QXD0068 - Reuso de Software

Universidade Federal do Ceará - *Campus* Quixadá

Prof. Francisco Victor da Silva Pinheiro
victorpinheiro@ufc.br



Agenda

- Pergunta
- Definição
- Origem
- Benefícios
- Obstáculos
- Características
- Fatores de Sucesso e Falha
- O Grande Problema
- Mitos, fatos e falácias
- Reuso ao redor do mundo
- Desenvolvimento chave em reuso de software
- O futuro do reuso de software
- Reuso de Software (2000–2010)
- Reuso de Software (2010–Atualidade)

Pergunta

- O que é reuso de software?



Pergunta

- Por que é importante para Engenharia de Software?



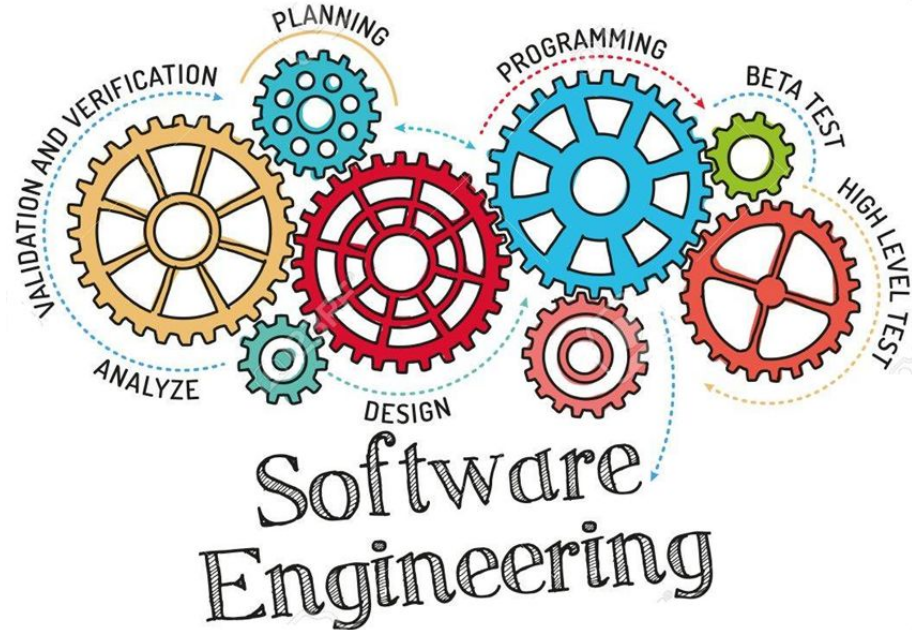
Resposta, talvez não a melhor

- **Reuso de software** é a prática de desenvolver novos sistemas aproveitando artefatos já existentes — como código, componentes, padrões ou conhecimento — em vez de construir tudo do zero.



Resposta, talvez não a melhor

- Ele é importante para a **Engenharia de Software** porque aumenta a produtividade, reduz custos e tempo de desenvolvimento, melhora a qualidade e a confiabilidade dos sistemas, além de favorecer a padronização e a manutenção ao longo do ciclo de vida do software.



Conexão com Engenharia de Software

- **Arquitetura de Software**
 - Reuso depende de design adequado para garantir extensibilidade e modularidade.
 - Arquiteturas baseadas em camadas, componentes e microserviços favorecem reuso.

- **Padrões de Projeto (Design Patterns)**
 - Catálogo do GoF é um marco: soluções reutilizáveis para problemas recorrentes de projeto.
 - Promovem padronização e comunicação entre desenvolvedores.

- **Linhas de Produto de Software (LPS)**
 - Estratégia de reuso sistemático em domínios específicos.
 - Permite desenvolver famílias de sistemas a partir de uma base comum de ativos.

Conexão com Engenharia de Software

- **Processos de Software**

- Modelos como RUP, CMMI e Agile incluem práticas que incentivam reuso.
- DevOps e CI/CD facilitam manutenção, testes e integração de componentes reutilizáveis.

- **Gerência e Qualidade de Software**

- Reuso impacta métricas de qualidade: confiabilidade, manutenibilidade, produtividade.
- Envolve também aspectos econômicos e organizacionais (custo-benefício, ROI, suporte da gerência).

- **Engenharia de Domínio**

- Identificação de similaridades entre aplicações.
- Criação de modelos e frameworks reutilizáveis para um domínio específico (financeiro, saúde, telecom).

Definição de Reuso

- Ao todo, são apresentadas 5 definições. São elas:
 - **Freeman:** É o uso de qualquer informação que o desenvolvedor pode precisar na criação de processo de software.
 - **Basili e Rombach:** O uso de tudo que esteja associado ao processo de desenvolvimento de software, incluindo o conhecimento.
 - **Frakes e Isoda:** O uso de engenharia de conhecimento ou artefatos de softwares existentes para a construção de novos softwares.

Definição de Reuso

- **Tracz:** O uso de software que foi projetado para reuso
- **Ezran:** A prática sistemática do desenvolvimento de software a partir de um conjunto de blocos, de forma que similaridades em termos de requisitos e/ou arquitetura entre aplicações possam ser exploradas para se alcançar substanciais benefícios em produtividade, qualidade e desempenho do negócio

Definição de Reuso

- Mas a adotada pelos autores é a seguinte:

“Reuso é o processo de criação de sistemas de software a partir de software já existente, em vez de construí-lo do zero.”

Origem

- **Idéia de reuso não é nova (1968)**
 - Já nos anos 60 discutia-se a necessidade de evitar retrabalho no desenvolvimento de software, inspirando-se em práticas de produção em massa de outras engenharias.
- **Conferência de Engenharia de Software (NATO)**
 - Foi o marco inicial da Engenharia de Software como disciplina.
 - Nesse evento, pesquisadores como Doug McIlroy defenderam a criação de bibliotecas de componentes reutilizáveis, destacando que o software deveria ser produzido como peças padronizadas de uma indústria.

Origem

- **Crise do Software**
 - Termo usado para descrever problemas da época: atrasos constantes, custos altos, baixa confiabilidade e dificuldade de manutenção.
 - O crescimento da demanda por software era muito maior do que a capacidade de entrega das equipes.
- **A indústria do software está fraca e um aspecto dessa franqueza é a falta de componente de software sub-industrial**
 - Faltavam componentes padronizados e reutilizáveis. Cada novo sistema era desenvolvido do zero, o que aumentava custos e diminuía a produtividade.

Origem

- **Ponto de partida para investigar técnicas de produção em massa do software**
 - A partir daí, surgiram esforços de pesquisa e prática voltados ao reuso sistemático de software, tentando aplicar a lógica da indústria manufatureira (produção em série, padronização e montagem de blocos) ao desenvolvimento de sistemas.
- O reuso nasceu como resposta à Crise do Software e como proposta de industrialização do desenvolvimento de software.
- Produção em massa de componentes de softwares (Doug McIlroy)

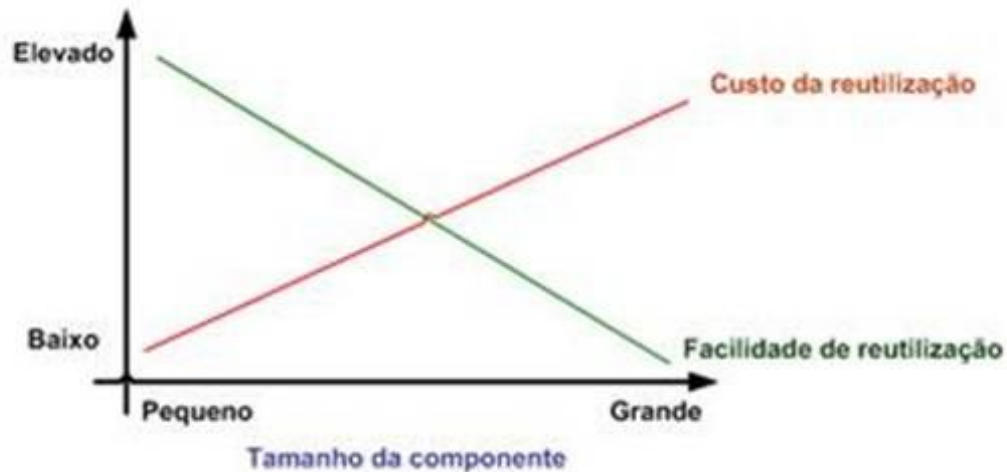
Existem Benefícios?

- Reutilizar traz algum benefício?



Existem Benefícios?

- Quais?



Existem Benefícios?

- Impactos positivos na qualidade do software, custo e produtividade
- Quanto a melhoria da qualidade:
 - Qualidade
 - Produtividade
 - Confiabilidade

Existem Benefícios?

- Quanto a redução do esforço
 - Redução do trabalho e o tempo de desenvolvimento
 - Tempo de Mercado
 - Documentação
 - Custo da manutenção
 - Tamanho da Equipe

Existem obstáculos?

- Existem obstáculos para se reutilizar um software?
- Quais?



Obstáculos

- **Podem ser do tipo:**
 - Gerencial
 - Organizacional
 - Econômico
 - Conceitual ou Técnico
- **Gerencial e Organizacional**
 - Falta de suporte de gerenciamento
 - Gerenciamento de Projeto
 - Estruturas organizacionais inadequadas

Obstáculos

- **Econômicos**
 - Leva mais tempo
 - É mais caro
 - “componentes devem ser reusados mais de 13 vezes para recuperar o investimento...” [Favaro,1991]
- **Conceitual e Técnico**
 - Dificuldade de encontrar software reusável
 - Não reusabilidade do software encontrado
 - Modificação

Características Básicas

- A área de reuso de software tem três características chaves
 - Prática sistemática de desenvolvimento de software.
 - Explora similaridade em requisitos e/ou arquiteturas entre aplicações.
 - Oferece benefícios substanciais na produtividade, qualidade e performance comercial.

O que reusar?

- Recursos de software reusáveis
- **Podem ser de dois tipos:**
 - Vertical: específico (software financeiro, médico, algoritmo)
 - Horizontal: genérico, podem ser utilizados independente da aplicação (autenticação de serviço)

Fatores de Sucesso e Falha

- **Problemas**
 - Não existe um conjunto de fatores de sucesso comuns entre empresas
 - Reuso é vantagem competitiva
- **Sucesso**
 - Gerenciamento
 - Fatores humanos
 - Maturidade do processo

Fatores de Sucesso e Falha

- Falha
 - 1/3 dos projetos falharam [Morisio, 2002]
 - Não introduzir processos específicos de reuso
 - Não modificar processos existentes que não consideravam reuso
 - Não considerar fatores humanos como parte do processo

O grande problema - ARIANE 5

- Em 4 de Julho de 1996, o foguete Ariane 5, da Agência Espacial Européia, explodiu 4 segundos depois do seu lançamento. Depois de uma década de desenvolvimento e mais de \$7 bilhões de investimentos.
- A causa da queda se deu por problemas no Sistema de Referência Inercial (SRI), que foi desenvolvido e reusado do Ariane 4.
- O fato do Ariane 5 ter uma aceleração inicial cinco vezes maior que o Ariane 4, fez com que ocorresse um overflow em um operação de conversão de tipos.
- Conclusão: é importante realizar uma avaliação completa de um componente em seu novo contexto [Weyuker, 1998]



Fatores de Sucesso e Falha

- Reuso é considerado uma vantagem competitiva
 - Pouca pesquisa é realizada
 - Fatores de sucesso são aplicáveis em todas as organizações?
 - Pesquisas informais
 - Pesquisas empíricas

Fatores de Sucesso e Falha

- Pesquisas informais
 - (Frakes e Isoda, 1994) Reuso sistemático é uma mudança de paradigma na Engenharia de Software, da construção de sistemas únicos para a construção de famílias de sistemas relacionados.
 - Frakes e Isoda listam 6 fatores críticos:
 - Gerência
 - Medições
 - Aspectos legais
 - Economia
 - Design para reuso
 - Bibliotecas

Fatores de Sucesso e Falha

- Pesquisas informais
 - Card e Comer (1994)
 - Fatores de fracasso
 - Reuso como aquisição de tecnologia
 - Reuso como uma estratégia de negócio (market driven)
 - Fatores críticos
 - Treinamento
 - Incentivos e medições
 - Gerência

Fatores de Sucesso e Falha

- Pesquisas informais
 - Glass (1998)
 - Reuso possui problemas na prática
 - Não existem muitos componentes de software que podem ser reusados
 - Velocidade e quantidade x Qualidade e reusabilidade
 - Projetos possuem cronogramas
 - Medições de software focam em quantidade
 - Reuso conceitual x Reuso de componentes

Fatores de Sucesso e Falha

- Pesquisas empíricas
 - Rine (1997)
 - Survey para investigar quais os fatores de sucesso comuns entre todos os domínios de aplicação
 - Fatores de sucesso
 - Abordagem de linha de produto
 - Arquitetura padronizam interfaces e formatos dos dados
 - Engenharia de domínio
 - Processo de reuso
 - Gerência que compreende os aspectos do reuso

Fatores de Sucesso e Falha

- Pesquisas empíricas
 - Rine (1997) - Fatores de sucesso (continuação)
 - Advogados do reuso na alta gerência
 - Experiência no reuso de artefatos de software de alto nível
 - Habilidade de rastrear os requisitos para os componentes
 - Critica a visão de que um repositório é uma condição necessária

Fatores de Sucesso e Falha

- Pesquisas empíricas
 - Morisio et al (2002)
 - Estudo mais detalhado em reuso de software disponível na literatura
 - Práticas de reuso em companhias européias: 24 projetos conduzidos entre 1994 e 1997
 - Companhias grandes e pequenas
 - Diversos domínios
 - Abordagens OO e procedurais
 - Por volta de $\frac{1}{3}$ dos projetos falharam

Fatores de Sucesso e Falha

- Pesquisas empíricas
 - Morisio et al (2002) – continuação
 - Fatores de fracasso:
 - Não introduzir processos específicos para o reuso
 - Não modificar processos tradicionais
 - Não considerar fatores humanos
 - Falta de comprometimento da alta gerência
 - Paradigma OO garante o reuso com sucesso
 - Repositório garante o reuso com sucesso

Fatores de Sucesso e Falha

- Pesquisas empíricas
 - Rothenberger et al (2003)
 - Sucesso varia com a estratégia aplicada?
 - Dados de 71 grupos de desenvolvimento
 - Entrevistados: gerentes, engenheiros de software, desenvolvedores, consultores e pesquisadores em vários domínios
 - Mais de 80% dos participantes trabalhavam em organizações com mais de 200 funcionários

Fatores de Sucesso e Falha

- Pesquisas empíricas
 - Rothenberger et al (2003) - continuação
 - Fatores de sucesso:
 - Planejamento
 - Formalização dos processos
 - Suporte da gerência
 - Similaridade dos projetos
 - Independente de técnicas OO

Mitos, fatos e falácias

- Frakes e Fox (1995) responderam 16 questões
- Survey que coletou dados de organizações dos EUA e Europa
 - Os artefatos reusáveis são amplamente reusáveis? (às vezes)
 - As linguagens afetam o reuso? (não)
 - As ferramentas CASE promovem o reuso? (não)
 - Os desenvolvedores preferem construir do zero ou reutilizar?
 - A educação em reuso influencia o reuso? (sim)
 - A experiência em Engenharia de Software influencia o reuso? (não)
 - Premiações e reconhecimento aumentam o reuso? (não)
 - Um processo de software comum promove o reuso? (provavelmente)

Mitos, fatos e falácias

- Frakes e Fox (1995) - continuação do Survey
 - Problemas legais inibem o reuso? (não)
 - Repositórios aumentam o reuso de código? (não)
 - O reuso é mais comum em determinadas indústrias? (sim)
 - O tamanho do projeto ou organização definem o reuso da organização? (não)
 - Aspectos da qualidade inibem o reuso? (não)
 - As organizações medem o reuso, a qualidade e a produtividade? (geralmente não)
 - As medições do reuso influenciam o reuso? (não)

Mitos, fatos e falácias

- Glass (2002) publicou 6 fatos sobre reuso de software
 - F1: Reuso “in-the-small” (bibliotecas de subrotinas) iniciou há aproximadamente 50 anos e é um problema bem resolvido
 - F2: Reuso “in-the-large” (componentes) permanece um problema não resolvido
 - F3: Reuso in-the-large” trabalha melhor em famílias de sistemas relacionados

Mitos, fatos e falácias

- Glass (2002) - Fatos sobre reuso de software - Continuação
 - F4: Existem duas “regras de três” em reuso
 - (i) É três vezes mais difícil construir componentes reusáveis
 - (ii) Um componente reusável deve ser utilizado em 3 aplicações diferentes antes de ser geral o suficiente para ser incluído em uma biblioteca
 - F5: Modificação de código reusado é uma fonte de erros
 - F6: Design Patterns são a solução para o problema inerente ao reuso de código

Reuso ao redor do mundo

- **STARS – Software Technology for Adaptable Reliable Systems**
 - 1988 – 1996: Mark Simos e pesquisadores de várias empresas
 - Produziu vários documentos e uma metodologia de reuso de software
 - Um conjunto de projetos foi desenvolvido até 1996
 - Organization Domain Modeling (ODM)
 - Reuse Library Framework (RLF)
 - Reuse-Oriented Software Evolution (ROSE)
- Resultados disponíveis para o uso público até 1999

Reuso ao redor do mundo

- **KOTEF – Korea Industrial Technology Foundation**
 - 1990: iniciativa do POSTECH Software Engineering Laboratory da Korea
 - Projetos de reuso de software (2003 - 2005)
 - Métodos e ferramentas para engenharia de linha de produtos
 - Lacuna de informações sobre as técnicas, métodos, processos e resultados adotados por esta iniciativa <http://selab.postech.ac.kr>
 - Iniciativa é referência em artigos técnicos e workshops da ciência da computação
 - Potencial da competitividade da Korea na indústria de software no futuro

Reuso ao redor do mundo

- **Iniciativas nos EUA**

- 1994 – 2000
- ATP - Advanced Technology Program - forneceu \$ 70 milhões de fundos públicos
- Indústria forneceu \$55 milhões de fundo-compartilhado
- Suporte a 24 projetos de desenvolvimento em tecnologia nas áreas de DBC, linha de produtos, entre outros
- 2/3 geraram produtos comerciais
- 3/4 alcançaram a fase de comercialização, apesar dos produtos ainda não terem gerado lucro

Reuso ao redor do mundo

- **Iniciativas de reuso de software na Europa**
 - Segundo MCGibbon et al (2001) existem alguns países da Europa (Inglaterra, Escandinávia, Alemanha e outros de regiões do norte) em que CBSE possui uma grande abrangência
 - Uma característica é que os clientes só confiam em componentes de software que tenham sido produzidos de acordo com padrões de controle de qualidade
 - CBSE: systems integrators, middleware e ferramentas de desenvolvimento

Reuso ao redor do mundo

- **Iniciativas de reuso de software no Brasil**
 - 2004: RiSE - Reuse in Software Engineering
 - Desenvolver um framework robusto para reuso de software em conjunto com o C.E.S.A.R
 - Projetos desenvolvidos em conjunto com universidades brasileiras

Desenvolvimentos chave em reuso

- McIlroy (1968): propôs investigar técnicas de produção em massa de software baseado nos padrões da indústria da construção
- Prieto-Diaz & Freeman (1987): apresentaram uma solução parcial para o problema da recuperação de componentes
- Krueger (1992) apresentou o primeiro survey importante na área de reuso de software
- Frakes & Fox (1995) apresentaram respostas de 16 questões recorrentes a partir de um survey aplicado nos EUA e Europa
- 1997 – 1999: reuso de software amadureceu
 - Livros na área: (Poulin, 1997), (Jacobson et al., 1997), Reifer (Reifer, 1997) e Lim (Lim, 1998).

Desenvolvimentos chave em reuso

- Relatório do SEI - Software Engineering Institute (Bass et al., 2000) levantou várias questões relacionadas à CBSE.
- Morisio et al. (2002) apresentou fatores chaves na adoção de um programa de reuso em uma organização a partir da análise de 24 projetos no período de 1994 a 1997.

Futuro do reuso de software

- Kim & Stohr (1998) - Direcionamentos para pesquisa
 - Medições
 - Metodologias
 - Ferramentas
 - Aspectos não-técnicos

Futuro do reuso de software

- Frakes & Kang (2005)
 - Problemas abertos
 - Programas de reuso
 - Estratégias para organizações
 - Medições
 - Metodologias
 - Bibliotecas
 - Confiabilidade
 - Segurança
 - Escalabilidade

Reuso de Software (2000–2010)

- **Software Product Lines (SPL):** consolidação da abordagem de Linhas de Produto de Software em setores como automotivo, aeroespacial e telecom. Empresas como Nokia, Siemens e GM investiram pesado na criação de plataformas de produto baseadas em ativos reutilizáveis.
- **Component-Based Software Engineering (CBSE):** foco em reuso via componentes e frameworks (ex.: COM, CORBA, EJB).
- **Avanços acadêmicos:** grandes conferências (ICSE, SPLC) passaram a ter trilhas específicas sobre reuso.
- **Programas de incentivo:** iniciativas governamentais como o DoD Software Reuse Initiative (EUA) evoluíram para projetos ligados a SPL.
- **Resultado prático:** empresas relatam ganhos de produtividade de 5x a 10x com SPL em domínios estáveis.

Reuso de Software (2010–Atualidade)

- **Microserviços e Arquitetura Cloud Native:** reuso passou a ocorrer na forma de serviços independentes, acessados por APIs (Amazon, Netflix, Google).
- **InnerSource:** cultura de reuso interno inspirado no open source, usada por Ericsson, Bosch, PayPal para compartilhar componentes e microserviços dentro da empresa.
- **DevOps + CI/CD:** automação tornou o reuso mais seguro, garantindo qualidade, testes e integração contínua de ativos.

Reuso de Software (2010–Atualidade)

- **Linhas de Produto ainda relevantes:** SPL continua aplicada em domínios embarcados e industriais, agora integrada com práticas ágeis.
- **Reuso assistido por IA:** ferramentas de ML/IA (e recentemente IA generativa) são usadas para detectar duplicações, sugerir componentes reutilizáveis ou adaptar código existente.
- **Estudos de caso recentes:**
 - Ericsson mostrou ganhos expressivos ao estruturar microserviços reutilizáveis em larga escala.
 - Pesquisas na Europa e Ásia apontam que o desafio atual não é só criar ativos reutilizáveis, mas garantir descoberta, governança e medição do reuso.

Linha do Tempo

Época	Abordagem dominante	Exemplos	Limitações
2000 – 2010	SPL, CBSE	Nokia, Siemens	Alto custo inicial
2010–2020	Microserviços, InnerSource	Netflix, Bosch	Governança, complexidade
2020 – Atual	IA, DevOps, Reuso generativo	GitHub Copilot, OpenAI, Google	Confiabilidade, aspectos legais

Resumo

- Reuso consiste em criar sistemas de softwares a partir de softwares já existentes.
- Essa estratégia proporciona vantagem competitiva às empresas.
- Mas também podem proporcionar grandes problemas caso o componente reusado não seja adequado ao novo software.

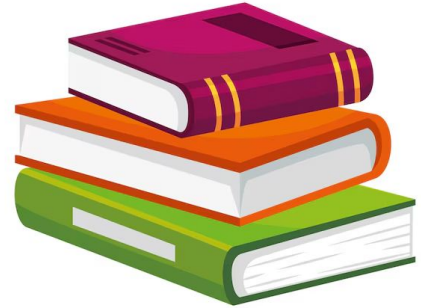
Bibliografia Básica

- MILI, Hafedh. Reuse-based software engineering: techniques, organization and measurement. New York: Wiley, 2002. 636 p. ISBN 0471398195.
- EZRAN, M.; MORISIO, M.; TULLY, C. Practical software reuse. Berlim: Springer, 2002
- POHL, K.; BÖCKLE, G.; LINDEN, F. J. Software product line engineering: foundations, principles and techniques. Berlim: Springer, c2010. xxvi, 467 p. ISBN 9783642063640 (broch.).



Bibliografia Complementar

- SOMMERVILLE, I. Engenharia de software. 7. ed. São Paulo: Pearson Addison-Wesley, 2007.
- PRESSMAN, R. Engenharia de software. 6. ed. São Paulo: Mc Graw-Hill, 2006. ISBN 8586804576
- GAMMA, E.; HELM, JOHNSON, R.; R.; VLISSIDES, J. Padrões de projeto: soluções reutilizáveis de software orientado a objetos. Porto Alegre: Bookman, 2000. 364 p.
- BRAUDE, Eric J.; BERNSTEIN, Michael E. Software engineering: modern approaches. 2nd ed. Hoboken, New Jersey: J. Wiley & Sons, 2011. xvi, 782 p.
- BASS, Len; CLEMENTS, Paul; KAZMAN, Rick. Software architecture in practice. 3rd ed. Upper Saddle River, NJ: Addison-Wesley, c2013. xix, 589 p. (SEI series in software engineering). ISBN 9780321815736 (enc.).



Obrigado! Dúvidas?



Universidade Federal do Ceará - *Campus* Quixadá

Prof. Francisco Victor da Silva Pinheiro
victorpinheiro@ufc.br

